

Санкт-Петербургский государственный университет
Кафедра системного программирования

Разработка байткода и виртуальной машины для языка Lata

Парфенов Данил Игоревич, группа 22.Б11-мм

Научный руководитель: доцент кафедры системного программирования, к. ф.-м. н., Булычев Д. Ю.

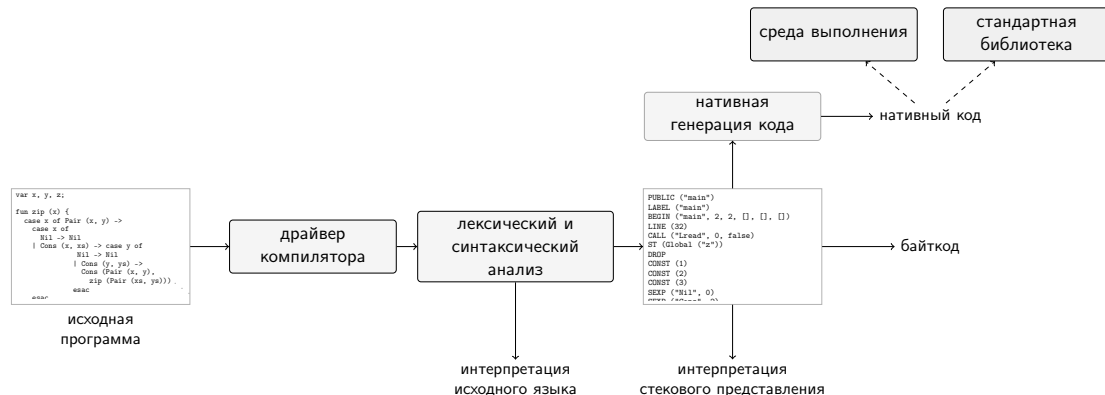
Консультант: аспирант кафедры системного программирования Доморацкий Э. А.

Рецензент: Инженер-программист 1 категории, АО "НПО Эшелон", к. ф.-м. н., Лозов П.А.

Санкт-Петербург
2026

- Язык программирования требует согласованной инфраструктуры: компилятора, промежуточных представлений и среды исполнения
- Байткод и виртуальная машина отделяют компиляцию от конкретного способа запуска программы
- Lata — учебный язык программирования, разработанный Дмитрием Булычевым для преподавания курса по разработке компиляторов.
- Компилятор Lata позволяет компилировать программы только в ассемблер архитектуры x86_64
- Виртуальная машина позволит студентам курса использовать Lata на практически любой платформе

Обзор инфраструктуры



- Компилятор поддерживает режим компиляции в байткод, но не существует стандартного интерпретатора для него
- На данный момент байткод неспецифированный и его запуск многомодульных программ из байткода технически невозможно реализовать

Постановка задачи

Цель: разработать виртуальную машину байткода языка программирования Lama, согласованную с существующим компилятором в ассемблер x86_64

Задачи:

- 1 Реализовать прототип виртуальной машины для существующего формата представления байткода Lama
- 2 Доработать формат представления байткода Lama и прототип виртуальной машины для возможности исполнения многомодульных программ
- 3 Реализовать виртуальную машину на основе прототипа с предварительным анализом байткода для ускорения выполнения
- 4 Проверить корректность работы виртуальной машины и оценить её производительность на существующих тестах

На первом этапе был реализован прототип виртуальной машины¹:

- Исходный байткод Lama задаёт стековую модель вычислений: инструкции берут аргументы со стека и помещают результат обратно на стек
- Значения в среде исполнения представлены в двух формах: `fixnum` для целых чисел и указатель на объект в куче для составных значений
- Составные объекты, строки, массивы и замыкания размещаются в куче
- Среда исполнения реализована на языке Си, поэтому виртуальная машина использует уже существующую инфраструктуру языка

¹<https://github.com/ancavar/Lama/pull/1>

Байткодное представление



Зеленым отмечены поля и секции, добавленные или измененные по сравнению с исходным форматом

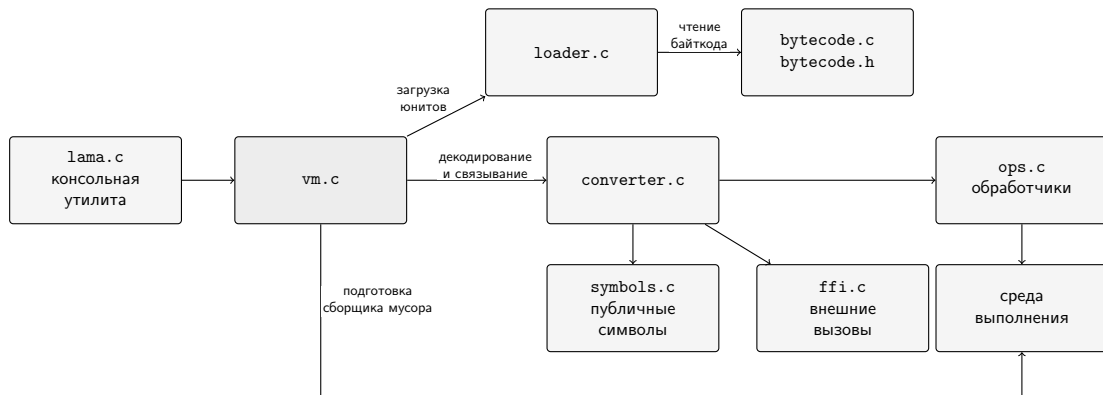
Новые записи

- добавлены магическое число и версия формата байткода
- `imports`: импортированный модули
- `publics`: добавлен флаг для различия публичных функций и глобальных переменных

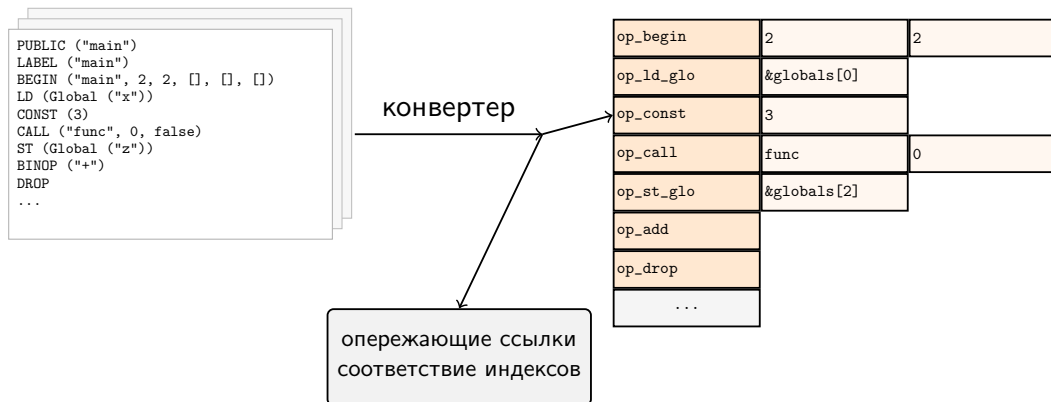
Изменения в инструкциях

- Добавлено количество замкнутых переменных в качестве операнда `BEGIN_CLOSURE`:
- Добавлена новая инструкция `FAIL_KEEP`
- Внешние ссылки кодируются отрицательными операндами через таблицу строк
- Вместо инструкций `read/write/length/string` вызываются непосредственно функции среды исполнения

Архитектура виртуальной машины



Подготовка кода



- Кодовая секция декодируется в один проход во внутреннее представление для каждого юнита
- Проверяются границы инструкций, цели переходов, вызовы и стековая глубина

Проверка корректности осуществлялась несколькими способами:

- Регрессионные тесты покрывают известные сценарии исполнения программ Lata
- Имеющийся компилятор Lata, запущенный на виртуальной машине, сравнивался с нативным кодом
- Фаззинг использовался для поиска ошибок в обработке байткода и исполнения программ

Конфигурация тестового оборудования:

- AMD Ryzen 7 6800H
 - ▶ каждый запуск привязывался к одному физическому ядру (без одновременной многопоточности)
- DDR5 16 ГБ @ 6400 МГц
 - ▶ без подкачки
- Ubuntu 26.04 x86_64
- gcc Ubuntu 15.2.0-16ubuntu1

RQ: Какова накладная стоимость виртуальной машины относительно нативного исполнения по времени и памяти на характерных нагрузках Lama?

Эксперимент: время работы

Набор тестов	нат.	прототип	BM	Замедление нат. vs BM	<i>n</i>
performance/Sort	28,57 с	95.5 с	48,85 с	1,71×	1
lama_compiler/regression	0,1907 с	0,3526 с	0,2526 с	1,33×	62
lama_compiler/regression/expressions	0,1187 с	0,1926 с	0,1465 с	1,25×	998
lama_compiler/regression/deep-expressions	3,3754 с	8,5804 с	4,8942 с	1,45×	11

- На всех рассмотренных вычислительно содержательных нагрузках new-vm медленнее нативного исполнения

Эксперимент: потребление памяти

Тест	нат.	прототип	BM
performance/Sort	200	195	214
lama_compiler/regression	65144	62052	63512
lama_compiler/expressions	31412	29796	29912
lama_compiler/deep-expressions	1038276	1091056	1124684

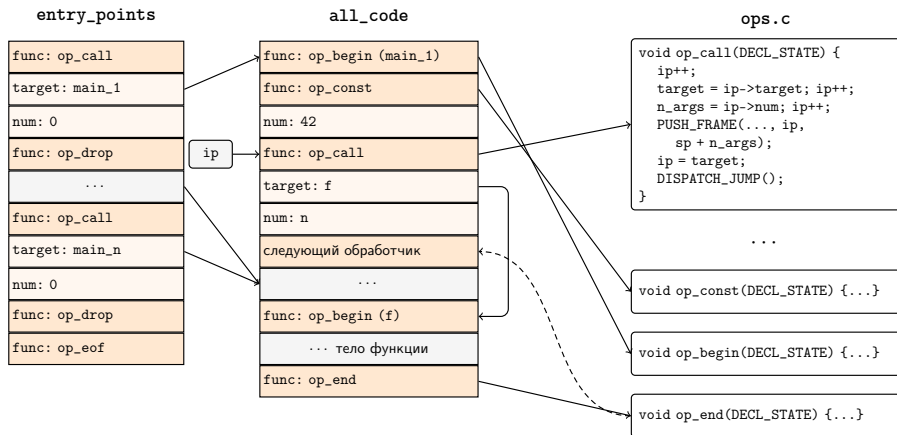
- В таблице приведён peak RSS для отдельных характерных тестов в килобайтах.
- RSS в пределах погрешности

В ходе данной работы были решены следующие задачи.

- 1 Реализован прототип виртуальной машины для существующего формата представления байткода Lama
- 2 Доработаны формат представления байткода Lama и прототип виртуальной машины для возможности исполнения многомодульных программ
- 3 Реализована виртуальная машина на основе прототипа с предварительным анализом байткода для ускорения выполнения
- 4 Проверены корректность работы виртуальной машины и её производительность на существующих тестах

Ознакомиться с кодом можно по ссылке

<https://github.com/ancavar/Lama/new/new-vm/>



- Внутреннее представление организовано как шитый код
- Стек виртуальной машины выделен отдельно с помощью mmap
- Внешние функции вызываются через интерфейс внешних функций